

The Non-Analytic Critical-Region Term

Part 4: Function-by-Function Walkthrough

outram-park-fork-coolprop

2026-07-10

Abstract

This document traces one concrete call — computing Water’s pressure at its own critical point — through every function it touches, from the public API down to the non-analytic term evaluator from Part 3, and shows the actual verified numerical result. Read alongside the source files named in each section; nothing here should require jumping to a file not named.

1 The entry point: `state_trho`

Everything starts from `props::state_trho(fluid, t, rho)` (`src/props.rs`) — the crate’s most basic query, “give me every property of this fluid at this temperature and density”:

```
let eos: &FluidEos = fluid.eos();
let delta = rho_molar / eos.rho_reducing;
let tau = eos.t_reducing / t;
let res = eos.residual_derivs(delta, tau);
let ide = eos.ideal_derivs(delta, tau);
// ... combine res, ide into p, u, h, s, cv, cp, speed_of_sound
```

For our example, `fluid = Fluid::Water`, and we choose `t` and `rho` to land *exactly* on Water’s own critical point: `t = 647.096` (Kelvin) and `rho =` the critical point’s mass density, `322.0 kg/m3` (obtained from `eos.rho_critical * eos.molar_mass` — `rho_critical` is molar, `17873.7 mol/m3`). Since Water’s reducing temperature and density *are* its critical values, this makes $\delta = \tau = 1$ exactly — the branch point Part 2 discussed.

2 Where Water’s data comes from: `fluid.eos()`

`fluid.eos()` (`src/fluid.rs`) is a `match` returning `&fluids::water::WATER` — a `const FluidEos` generated by `dev/gen_fluid.py` from CoolProp’s own `Water.json` (`upstream_source/CoolProp/dev/fluids/`). Its residual term list includes exactly one `ResidualTerm::NonAnalytic` entry, carrying the two-term coefficient arrays from Part 1 (`a: &[3.5, 3.5]`, `b: &[0.85, 0.95]`, `beta: &[0.3, 0.3]`, `big_a: &[0.32, 0.32]`, `big_b: &[0.2, 0.2]`, `big_c: &[28.0, 32.0]`, `big_d: &[700.0, 800.0]`) verbatim from CoolProp’s fitted values.

3 `eos.residual_derivs`: summing every term type

`FluidEos::residual_derivs(delta, tau)` (`src/eos.rs`) walks *every* entry in `self.residual` and calls `term.accumulate(delta, tau, &mut acc)` on each — for Water this means one `Power`-variant call (51 monomial terms), one `Gaussian`-variant call (3 bell-shaped terms), and the `NonAnalytic`-variant call from Part 3. All three add into the same accumulator; `residual_derivs` has no idea, and does not need to know, which term types actually contributed.

4 Inside the non-analytic call

With $\delta = \tau = 1$ exactly, `accumulate_non_analytic`'s guard (Part 2) fires on both variables, offsetting each by $10\epsilon \approx 2.22 \times 10^{-15}$. For term $i = 1$ ($a_1 = 3.5$, $b_1 = 0.85$, $\beta_1 = 0.3$, ...): $dm1 = \delta - 1 \approx 2.22 \times 10^{-15}$, so $dm1_sq \approx 4.9 \times 10^{-30}$ — an extremely small but perfectly finite `f64`. Propagating this through θ , Δ , Δ^{b_i} and ψ (Part 2's chain) and their derivatives gives six finite numbers per term; both terms' contributions are summed into `acc` via `add_scaled`.

5 Back up the stack: assembling the pressure

Once `residual_derivs` returns, `state_trho` combines the residual α_δ (called `res.ad` in code) with the ideal-gas part via the standard Span–Wagner relation

$$p = \rho R T (1 + \delta \alpha_{r,\delta}),$$

implemented directly as `rho_molar * r * t * (1.0 + delta * res.ad)`.

6 The verified result

`tests/non_analytic_critical_region.rs` runs exactly this computation and asserts the result against IAPWS-95's defining critical pressure:

```
let s = state_trho(Fluid::Water, eos.t_critical, rho_mass_c);
let rel = (s.pressure - eos.p_critical).abs() / eos.p_critical;
assert!(rel < 1e-9);
```

Running it (`cargo test --release -p outram-park-fork-coolprop --test non_analytic_critical_region -- --nocapture`) prints:

```
p(Tc, rho_c) = 22064000.00000 Pa (p_c = 22064000 Pa), rel err = 5.217e-14
```

five orders of magnitude tighter than the 10^{-9} gate, and — as documented in this crate's git history — a substantial improvement over the $\sim 10^{-4}$ residual the same computation carried before `accumulate_non_analytic` was implemented (when the `NonAnalytic` match arm was a documented no-op). The full verification record, including the reference BibTeX citation with page number, lives in `verification_and_validation/water_critical_point_iapws95.md`.